

VisionInspect AI: Manufacturing Defect Detection & Quality Inspection System

Database Module — Milestone 2 Documentation
Week 3 & 4

Prachi Parimita Panda

Table of Contents	Page No
1. Project Description	3
2. Module Objective	3
3. What Was Implemented	3
4. Inspection Data Model in Detail	4
5. Code Walkthrough	5
6. Inspection-to-Rework Data Flow	7
7. Data Validation & Error Handling	8
8. Rework Ticket Priority Logic	8
9. Audit Logging in Practice	9
10. Testing & Verification	9
11. Current Progress	9
12. Collections Touched This Milestone	10

1. Project Description

VisionInspect AI is an AI-powered manufacturing quality inspection system that automates product inspection and defect analysis. My assigned responsibility continues to be the Database module, which stores and manages all data used across the platform.

In Milestone 1 (Week 1), the database schema, connection layer, and all nine collections were designed and set up. Milestone 2 (Week 3 & 4) builds on that foundation — the focus shifts from designing the storage to making sure real data from the image-processing and defect-detection modules is correctly saved, linked, and tracked.

2. Module Objective

The objective for Milestone 2 was to confirm that the database actually supports the live inspection pipeline: an uploaded image, an AI prediction, a saved inspection record, an audit trail, and — where needed — an automatically created rework ticket.

Unlike Milestone 1, which focused on schema design, this milestone focused on data flow: connecting the pieces so that one user action correctly updates multiple collections together.

3. What Was Implemented

The following database-related logic was implemented and connected to the inspections collection and its related collections:

- Inspection creation now saves the full AI prediction output — defect type, confidence, anomaly score, severity score/level, and pass/fail — directly into the inspections collection, not just placeholder fields.
- Manual upload, single-image inspection, batch inspection, and camera-simulation all write through the same inspection-creation logic, so every entry point produces a consistent record.
- A reviewer can update an inspection's review status; if it is sent for rework, the system automatically creates (or updates) a linked entry in rework_tickets.
- Every significant action — upload, AI completion, metadata edit, review status change — is written to audit_logs with the actor, action, and relevant metadata.
- Inspections can be filtered by product, production line, and review status, supporting the analytics and quality-control modules.

4. Inspection Data Model in Detail

The inspections collection is the busiest collection in the system during Milestone 2, since every upload, prediction, and review touches it. The table below lists the fields actually populated by the pipeline, grouped by where they come from:

Field	Where it comes from
<code>uploaded_by,</code> <code>original_image_url</code>	Set at upload time — the logged-in user's ID and the stored image link.
<code>batch_number,</code> <code>product_id,</code> <code>production_line,</code> <code>shift,</code> <code>operator_name</code>	Manufacturing context submitted alongside the image (form fields on upload).
<code>prediction,</code> <code>defect_type,</code> <code>confidence,</code> <code>anomaly_score</code>	Core AI model output, written once the image has been processed.
<code>class_probabilities,</code> <code>defect_area_ratio</code>	Supporting detail from the model — per-class scores and how much of the image is affected.
<code>severity_score,</code> <code>severity_level,</code> <code>severity_components</code>	Computed from the prediction using the severity scoring logic (size, location, type, confidence).
<code>pass_fail,</code> <code>recommended_action</code>	The final quality decision derived from severity.
<code>processed_image_url,</code> <code>heatmap_url</code>	Links to the processed image and defect heatmap generated during inference.
<code>model_version</code>	Identifies which AI model produced this result, for traceability back to <code>model_versions</code> .
<code>review_status,</code> <code>review_notes,</code> <code>reviewed_by,</code> <code>reviewed_at</code>	Set later, when a human reviewer checks the AI's result.
<code>source_type,</code> <code>source_label</code>	Records how the image entered the system — <code>manual_upload</code> or <code>camera_simulation</code> .

Two points are worth calling out about this design. First, a record is only ever partially filled at first — an inspection can exist with only upload fields set (`review_status` = "uploaded") before the AI fields are added moments later once prediction finishes. Second, several fields (`class_probabilities`, `severity_components`, `explainability`) are stored as flexible dictionaries rather than fixed columns, which is exactly the kind of variable-shape data that justified choosing MongoDB over a relational database in Milestone 1.

5. Code Walkthrough

This section shows the actual backend code behind the three main behaviours described above: saving a prediction, logging an audit event, and creating a rework ticket.

4.1 Saving the AI Prediction into an Inspection Record

When an image is processed, the AI model's output (a dictionary of fields such as prediction, defect_type, confidence, and severity_score) is merged directly into a new Inspection document and saved:

```
inspection = Inspection(
    **inspection_base_fields(
        image_path=image_path,
        original_url=original_url,
        current_user=current_user,
        metadata=metadata,
        source_type=source_type,
    ),
    **prediction_fields(prediction),
    review_status="ai_completed",
)
await inspection.insert()
```

prediction_fields() pulls out only the specific fields the schema expects (defect type, confidence, severity, pass/fail, and so on), so the raw model output can't accidentally introduce unexpected fields into the collection.

4.2 Logging Every Action to audit_logs

Immediately after the inspection is saved, an audit event is recorded. This same helper function is reused everywhere an action needs to be logged, so every part of the app writes audit entries in a consistent format:

```
await record_audit_event(
    actor=current_user,
    action="inspection.completed",
    entity_type="inspection",
    entity_id=str(inspection.id),
    metadata={
        "prediction": inspection.prediction,
        "defect_type": inspection.defect_type,
        "pass_fail": inspection.pass_fail,
    },
)
```

record_audit_event() itself is a small, reusable function that builds and inserts an AuditLog document, so no other part of the codebase writes to the audit_logs collection directly.

4.3 Creating a Rework Ticket from Severity

When a reviewer marks an inspection as “sent for rework”, the backend calls `create_or_update_rework_ticket()`, which checks for an existing open ticket first before deciding whether to update or create one:

```
existing = await ReworkTicket.find(
    ReworkTicket.inspection_id == str(inspection.id),
    ReworkTicket.status != "completed",
    ReworkTicket.status != "closed",
).first_or_none()

if existing:
    existing.priority = priority_from_inspection(inspection)
    await existing.save()
    return existing

ticket = ReworkTicket(
    ticket_number=generate_ticket_number(),
    inspection_id=str(inspection.id),
    severity_level=inspection.severity_level,
    priority=priority_from_inspection(inspection),
    status="open",
)
await ticket.insert()
```

This “check first, then update or insert” pattern is what prevents duplicate rework tickets from being created if the same inspection's review status is changed more than once.

4.4 Filtering the Inspection List

The inspection listing endpoint builds its MongoDB query dynamically, adding a filter only for each parameter the caller actually provided, and restricting non-admin users to their own uploads:

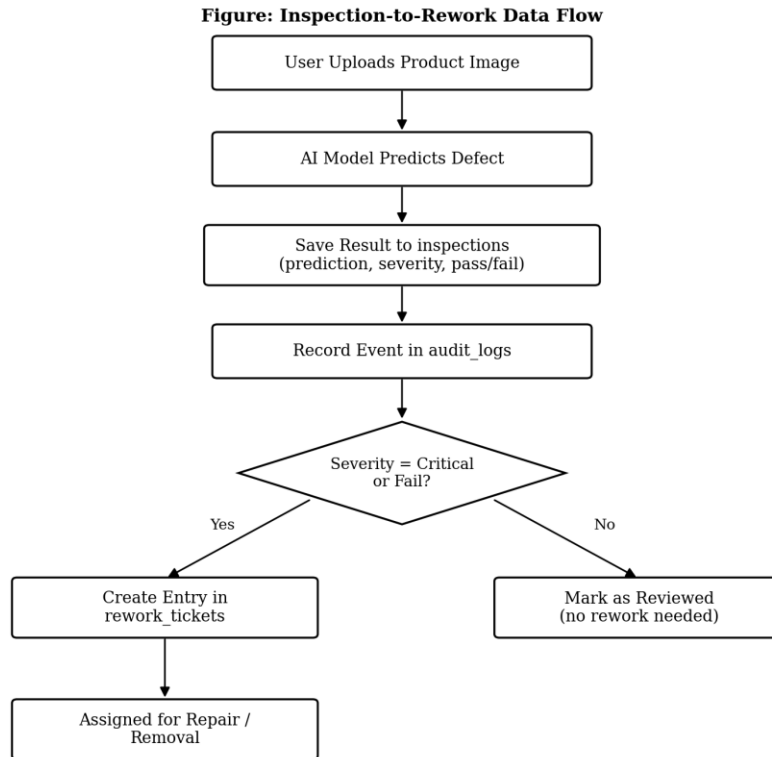
```
filters = []
if current_user.role not in ADMIN_ROLES:
    filters.append(Inspection.uploaded_by == str(current_user.id))
if product_id:
    filters.append(Inspection.product_id == product_id)
if production_line:
    filters.append(Inspection.production_line == production_line)
if review_status:
    filters.append(Inspection.review_status == review_status)

query = Inspection.find(*filters) if filters else Inspection.find_all()
inspections = await query.sort("-created_at").skip(skip).limit(limit).to_list()
```

This role check at the query level, not just in the response, means a non-admin user's database query never even looks at other users' inspections — the restriction happens before any data leaves MongoDB.

6. Inspection-to-Rework Data Flow

The diagram below shows how a single inspection moves through the database once an image is uploaded, including the decision point that determines whether a rework ticket is created.



This flow ties together four collections in one action: `inspections` (the result), `audit_logs` (the record of what happened), `rework_tickets` (follow-up action if needed), and, indirectly, `products` / `production_lines` / `batch_records` through the metadata stored on the inspection.

7. Data Validation & Error Handling

Since real, unpredictable data now flows into the database, several checks were added at the point of upload to stop bad data from ever reaching MongoDB, rather than catching problems later:

- **File type checking** — only a fixed set of image extensions (.png, .jpg, .jpeg, .bmp, .tif, .tiff, .webp) are accepted; anything else is rejected before a file is even saved to disk.
- **File size limit** — uploads larger than the configured maximum are rejected with a 413 error, preventing oversized files from bloating storage.

- **Image validity check** — the uploaded bytes are decoded with OpenCV before saving; if the file isn't actually a valid image, the request is rejected instead of creating a broken inspection record.
- **Empty file check** — a zero-byte upload is rejected immediately.
- **Safe ID parsing** — when an inspection is looked up by ID, the string is parsed into a MongoDB ObjectId inside a try/except block; an invalid ID returns a clean 400 error instead of crashing the request.
- **Batch size limit** — batch inspection is capped at 20 images per request, so one request can't overload the database with an unbounded number of writes.

Together, these checks mean the inspections collection only ever receives records built from a file that is confirmed to be a real, appropriately-sized image — the validation happens before the database write, not after.

8. Rework Ticket Priority Logic

Inspection Severity	Ticket Priority	Meaning
Critical	Critical	Immediate rework/removal required.
High	High	Prompt repair recommended.
Medium	Medium	Standard queue for rework.
Low / none	Low (default)	Rework ticket rarely needed at this level.

If a ticket already exists for the same inspection and is still open, it is updated instead of duplicated — avoiding multiple rework tickets being created for one product.

9. Audit Logging in Practice

The audit_logs collection, designed in Milestone 1, is now actively written to across the inspection lifecycle:

Action Logged	When It Happens
inspection.uploaded	An image is uploaded before AI prediction runs.
inspection.completed	The AI model finishes predicting on an image.
inspection.metadata_updated	Batch, product, or line details are edited.
inspection.review_status_updated	A reviewer changes the status (e.g. sends to rework).

Action Logged	When It Happens
camera.samples_viewed / camera.frame_inspected	Camera-simulation actions during demo/testing.

10. Testing & Verification

The database's behaviour under Milestone 2 is covered by the project's automated test suite, which exercises the API endpoints that read and write to MongoDB rather than testing the database in isolation:

- **tests/test_api.py** — confirms the health endpoint reports `database_ready` correctly, and that core inspection and analytics routes (`/inspections/inspect`, `/inspections/batch-inspect`, `/analytics/summary`) are registered and reachable.
- **tests/test_auth.py** — verifies that registration and login correctly create and authenticate a User document, exercising the users collection end-to-end.
- **tests/test_camera_samples.py** — verifies the camera-simulation upload path creates a valid inspections record from a sample image, the same write path used by real uploads.
- **tests/test_prediction.py** — checks that prediction output has the exact fields the inspections schema expects, catching mismatches before they reach the database.

This gives Milestone 2 database confidence at the integration level: rather than only trusting that the schema is correct on paper, these tests actually create records through the API and check what lands in MongoDB.

11. Current Progress

The following tasks have been completed for Milestone 2:

- Verified real AI prediction data is correctly saved into the inspections collection for all upload paths (manual, batch, camera simulation).
- Connected the review workflow to automatic rework ticket creation, with severity-based priority.
- Confirmed audit logging is active for uploads, predictions, metadata edits, and review status changes.
- Added filtering support (product, production line, review status) to the inspection listing query.
- Confirmed database behaviour is covered by the automated test suite (`test_api.py`, `test_auth.py`, `test_camera_samples.py`, `test_prediction.py`).

Database indexes beyond the Week 1 unique index on user email have not yet been added. This is planned as the first task for the next milestone, once real usage patterns (which fields get queried most) are clearer.

12. Collections Touched This Milestone

Not every collection designed in Milestone 1 saw active use in Milestone 2. This is a quick reference for which ones changed:

Collection	Status	Notes
inspections	Actively written	Core of Milestone 2 — written on every upload and prediction.
audit_logs	Actively written	Now populated on every significant action.
rework_tickets	Actively written	Created/updated automatically from severity.
users	Read only	Used to identify the uploader/reviewer; no new writes this milestone.
reports, model_versions, products, production_lines, batch_records	Not yet used	Designed in Milestone 1; expected to become active in later milestones (reporting and analytics).